SPIMP-RAG: Structure-Prior Injected Message Passing for Triple Retrieval in Knowledge Graph Question Answering

Jun Chen[a,1], Zhi-jun Xie[a,*]

[a]Faculty of Electrical Engineering and Computer Science, Ningbo University, Ningbo, 315211, China.

ARTICLE INFO

Keywords:
Knowledge graph question answering
Retrieval-augmented generation
Triple retrieval
Graph neural networks
Weak supervision.

ABSTRACT

Knowledge graph question answering (KGQA) with large language models (LLMs) relies on retrieving a compact set of supporting triples under strict context budgets. However, structure-free retrieval can easily return semantically relevant yet structurally inconsistent evidence, which degrades evidence quality under tight budgets and can hurt downstream reasoning. We propose SPIMP-RAG, a graph-grounded framework that centers on a coarse-to-fine triple retrieval pipeline with structural inductive bias. Starting from a question-centered candidate subgraph, a lightweight DDE+MLP coarse retriever produces a high-recall triple set, which is then refined by Structure-Prior Injected Message Passing (SPIMP) that injects Directional Distance Encoding (DDE) into relation-aware message passing and triple reranking. We further introduce adaptive semantic-structural fusion and a confidence-weighted weak-supervision scheme to train the retriever from question–answer pairs. We evaluate retrieval effectiveness and end-to-end KGQA on WebQSP and ComplexWebQuestions, focusing on the low-K regime where evidence quality is critical for truth-grounded reasoning. For module-level ablations, we report AER@50 as a stable moderate low-budget operating point, while strict low-K behavior is supported by full retrieval curves at smaller K values.

1. INTRODUCTION

Large language models (LLMs) have shown strong capabilities in natural language understanding and reasoning, yet knowledge-intensive tasks still require reliable grounding in external evidence. In knowledge graph question answering (KGQA), a natural solution is to retrieve a small set of supporting triples from a knowledge graph (KG) and condition an LLM on these triples. A key practical bottleneck is the *low-K evidence regime*: due to strict context and retrieval budgets, the *Top-K* retrieved triples must be not only individually relevant, but also *jointly* informative and structurally coherent. When K is small, minor ranking errors can break topic-to-answer connectivity, yielding evidence that looks plausible in isolation but fails to support multi-hop reasoning.

Existing KGQA systems integrate KGs and LLMs in several ways. Planning or agentic frameworks generate relation-path plans or traverse the KG iteratively to retrieve evidence [1, 2]. While these approaches can exploit KG structure, they often require multiple LLM interactions or explicit search procedures, and can be sensitive to intermediate planning and pruning errors. Another line of work leverages GNNs to process question-specific subgraphs and extracts structured evidence for LLM-based answer prediction [3, 4]. Such methods can incorporate multi-hop graph context, but applying message passing over dense candidate subgraphs can introduce noise diffusion and adds nontrivial computation. Finally, lightweight triple retrievers score candidate triples in parallel using semantic similarity and simple topic-centric structural features [5]. These retrievers are efficient and often high-recall, yet reliably producing an information-dense and structurally consistent *Top-K* set remains challenging in the low-K regime, since triples are commonly evaluated with limited contextualization of their roles within the candidate subgraph.

In this work, we target this low-K ranking bottleneck and propose **SPIMP-RAG**, a coarse-to-fine triple retriever for KGQA. Given a question-centered candidate subgraph, we first apply a lightweight DDE+MLP coarse scorer to construct a compact, high-recall candidate set. We then rerank these candidates with Structure-Prior Injected Message Passing (SPIMP), which injects Directional Distance Encoding (DDE) as a topic-centric structural prior into relation-aware message passing. To prevent over-reliance on potentially noisy structural cues, we introduce Adaptive semantic-structural Fusion (ASSF) as a gated residual update. We train SPIMP-RAG from question–answer pairs with a confidence-weighted weak supervision scheme derived from topic-to-answer connectivity, and adopt a two-stage protocol that avoids backpropagation through discrete Top-K selection.

Our main contributions are summarized as follows:

- We formulate and target the low-K evidence retrieval regime in KGQA, and propose a coarse-to-fine triple retrieval pipeline that improves Top-K evidence quality under strict budgets.
- We propose SPIMP, a structure-prior injected, relation-aware message passing mechanism for triple reranking, together with ASSF for adaptive semantic-structural fusion.
- We develop confidence-weighted surrogate supervision and a two-stage training protocol that trains both coarse and fine retrievers from question–answer pairs without gold reasoning paths.

*Corresponding author
xiezhijun@nbu.edu.cn (Z. Xie)

- We evaluate retrieval quality and end-to-end KGQA on WebQSP and ComplexWebQuestions (CWQ) under varying retrieval budgets with fixed LLM reasoners.

Unless otherwise stated, we keep a low-K focus throughout; AER@50 is used only as a stable ablation operating point, while strict low-K trends are assessed with full retrieval curves at smaller K.

2. RELATED WORK

Surveys and unifying perspectives. Recent surveys and roadmaps summarize the evolution of complex KBQA as well as the emerging integration of LLMs with structured knowledge and graphs [6, 7, 8, 9]. Our work fits into the KG-RAG setting, where the core challenge is to retrieve a compact set of KG triples that both match the query and preserve multi-hop connectivity under strict evidence budgets.

KGQA and subgraph-centric multi-hop reasoning. Classical KBQA systems rely on semantic parsing and structured query execution on large KGs such as Freebase [10, 11, 12]. To better handle compositional multi-hop questions, later work generates question-specific query graphs or skeleton-based logical forms and executes them against the KG [13, 14, 15]. In parallel, many approaches retrieve a question-specific subgraph and perform reasoning on the induced structure [16, 17, 18, 19]. SQALER argues that multi-hop retrieval can be decoupled from logical reasoning for scalability [20], and generation-augmented iterative ranking further improves multi-hop KGQA by combining generation with candidate reranking [21]. These directions highlight that retrieval is not merely a recall problem: under strict evidence budgets, the retrieved facts must be jointly informative and preserve topic-to-answer connectivity.

Graph neural reasoning and graph-aware retrieval. Graph neural networks (GNNs), including GCN [22], GAT [23], and relational variants such as R-GCN [24], provide a natural tool to encode multi-relational neighborhoods for KG reasoning. They have been widely used in KGQA either as the main reasoning module or as a structure-aware retriever [25, 18, 26]. Scalable neighborhood aggregation enables efficient contextual encoding on large graphs [27]. Recent work further improves structural expressivity by injecting explicit structural signals, such as distance encodings and node labeling, into message passing [28, 29]. Neural tree-search style reasoning further combines structured expansion with learned propagation for multi-hop KGQA [30]. In the KG+LLM setting, GNN-RAG [3] and related GNN-enhanced retrievers extract graph evidence to support downstream LLM answering [31]. While effective, applying message passing on large, noisy candidate graphs can be computationally heavy and may diffuse noise, making fine-grained ranking under tight Top-K budgets challenging.

LLM-guided planning and traversal on KGs. A prominent line treats the LLM as a planner or agent that explores the KG through iterative expansion and pruning [1, 2, 32]. ORT introduces ontology-guided reverse thinking to navigate large search spaces for abstract targets [33], and graph expansion and pruning methods further aim to reduce the exploration space [34]. More broadly, chain-of-thought prompting and graph-augmented CoT variants encourage explicit multi-step reasoning and graph-grounded reasoning traces [35, 36, 37]. These approaches yield interpretable reasoning traces, but they typically require multi-step interaction loops and optimize for path-level exploration, which increases latency and can accumulate errors from intermediate plans or pruning decisions.

LLM-based KBQA via structured queries and interaction. Beyond traversal, LLMs have been used as structured query generators or interactive agents for KBQA. StructGPT and UniKGQA translate natural language questions into executable structured queries or unified retrieval–reasoning representations [38, 39]. Knowledge-augmented prompting and few-shot in-context learning further explore how to adapt LLMs to KGQA with limited supervision [40, 41, 42]. Interactive frameworks and chat-style KBQA further leverage multi-turn clarification and "generate-then-retrieve" routines to improve answerability [43, 44]. KG-to-text pipelines retrieve KG evidence and rewrite it into model-friendly text before answering [45]. These paradigms emphasize controllability and interaction, but still leave open the problem of selecting a compact, information-dense triple set under strict evidence budgets.

KG-RAG and subgraph retrieval under tight budgets. Retrieval-augmented generation grounds LLM outputs in external evidence and improves factuality on knowledge-intensive tasks [46, 47, 48, 49]. In practice, long-context limitations further motivate retrieving a small, information-dense evidence set rather than relying on large raw contexts [50]. Graph-RAG surveys and systems explore retrieving structured graph evidence for generation across tasks [9, 51]. In the KG-RAG setting, SubgraphRAG shows that parallel triple scoring with topic-centric structural signals can be highly effective [5]. Graph-RAG frameworks such as GrAG and G-Retriever further explore retrieving structured graph evidence for generation [52, 53]. However, when K is small, independently scoring triples provides limited contextualization for resolving fine-grained ranking among many semantically plausible candidates.

3. METHODOLOGY

This section details **SPIMP-RAG**, our triple-level Knowledge Graph (KG) retriever which integrates structural inductive biases into Graph Neural Network (GNN) message passing. Designed specifically for the KG-based Retrieval-Augmented Generation (RAG) setting, the retriever, given a natural language question q, assigns scores to candidate triples within a question-specific subgraph and returns the Top-K triples as concise evidence for downstream Large Language Model (LLM) reasoning. Unlike conventional GNN retrievers that primarily focus on answer-node scoring, our approach leverages a GNN as a contextual encoder

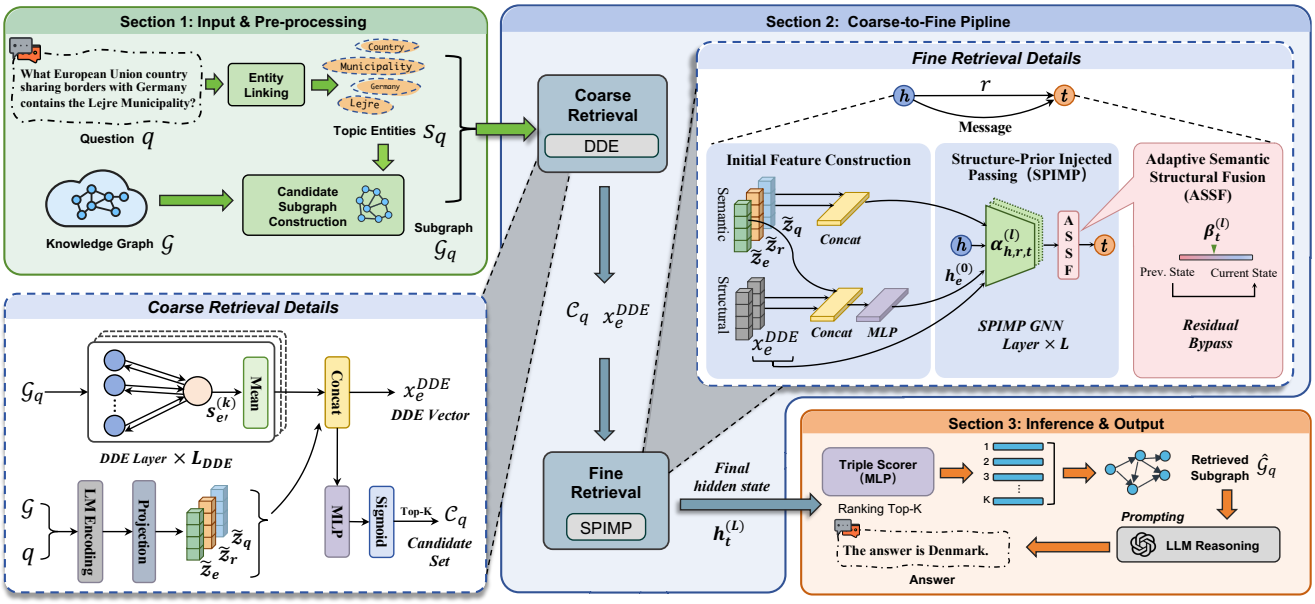

Figure 1: Overview of the SPIMP-RAG framework. A lightweight DDE+MLP coarse scorer selects a compact high-recall candidate set from a question-centered subgraph, which is then reranked by SPIMP with Adaptive semantic-structural Fusion (ASSF) to produce Top-K evidence triples for downstream LLM reasoning.

for entities, followed by query-aware triple scoring. This methodology maintains the efficiency and flexibility of parallel triple retrieval while enhancing structural faithfulness through Directional Distance Encoding (DDE). Crucially, our objective is not to supplant lightweight DDE+MLP retrieval, which is already highly effective for high-recall candidate generation, but to elevate the quality and information density of top-ranked triples within a constrained retrieval budget, a critical factor for effective LLM reasoning under context limitations. Fig. 1 provides an overview of the SPIMP-RAG framework.

3.1. Problem Definition

Let the knowledge graph be denoted as $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{T})$, where each triple is $\tau = (h, r, t) \in \mathcal{T}$. For a given natural language question q, we first perform entity linking to identify a set of topic entities, $\mathcal{S}_q \subseteq \mathcal{E}$. Subsequently, a question-specific rough candidate triple set $\mathcal{G}_q \subseteq \mathcal{T}$ is provided by preprocessing around $\mathcal{S}_q$.

3.1.1. Candidate Subgraph Construction

All retrievers in this work operate on a fixed question-specific rough candidate subgraph $\mathcal{G}_q$ provided by preprocessing around the linked topic entities $\mathcal{S}_q$. In our experiments, we reuse question-centered rough subgraphs released by prior work [5] as fixed preprocessing artifacts, and denote them uniformly as $\mathcal{G}_q$. Unless otherwise noted, both training and inference are performed under the same candidate-graph protocol. In this work, a subgraph is treated as a subset of KG triples; thus, $\tau \in \mathcal{G}_q$ indicates that triple τ is included in the candidate subgraph.

Our method does not introduce a new learned rough candidate generator. Instead, we assume $\mathcal{G}_q$ is given by preprocessing and focus on retrieval and reranking within this fixed candidate space. This design matches the role of $\mathcal{G}_q$ in the rest of our pipeline: the coarse retriever scores triples in $\mathcal{G}_q$ to construct a compact high-recall set, and the fine retriever further reranks this set with SPIMP.

To provide a lightweight semantic prior that is also reused later in weak supervision, we assign each candidate triple $\tau = (h, r, t)$ a fixed semantic relevance score

$$s_{\text{sem}}(\tau) = \cos(z_q,\ z_h + z_r + z_t), \tag{1}$$

where z_q is the frozen semantic embedding of the query and z_h, z_r, z_t denote the frozen embeddings of the head entity, relation, and tail entity, respectively, as defined in Sec. 3.2.1. Regardless of how the rough candidate subgraph is obtained, we treat $\mathcal{G}_q$ as a fixed discrete preprocessing artifact and do not backpropagate through its construction.

Our retrieval objective focuses on triples rather than individual nodes. Specifically, given $\mathcal{G}_q$, we aim to retrieve a compact set of question-relevant triples as evidence for downstream LLM reasoning. Under our coarse-to-fine formulation, we first apply a lightweight DDE+MLP coarse retriever on $\mathcal{G}_q$ to obtain a compact high-recall candidate triple set, and then refine it through SPIMP-based reranking, producing a final retrieved evidence subgraph that is linearized and provided to the LLM.

We next describe the semantic and structural feature representations shared by both coarse and fine retrieval stages.

3.2. Initial Feature Representations

For a given query q and candidate subgraph $\mathcal{G}_q$, we construct initial entity representations by integrating semantic embeddings and structural encodings.

3.2.1. Semantic Embedding

We employ a pre-trained language model (kept frozen) to encode the raw text of the query q, generic entities $e \in \mathcal{E}$, and relations $r \in \mathcal{R}$. This process yields dense semantic vectors z_q, z_e, and z_r, respectively. We treat these embeddings as fixed features and only learn the subsequent lightweight projection and retriever parameters. To ensure dimensional consistency for subsequent message passing and scoring, all semantic embeddings are projected into a shared hidden space:

$$\tilde{z}_q = W_q z_q, \quad \tilde{z}_e = W_e z_e, \quad \tilde{z}_r = W_r z_r, \tag{2}$$

where W_q, W_e, W_r are learnable projection matrices and $\tilde{z}_q, \tilde{z}_e, \tilde{z}_r \in \mathbb{R}^d$ share a common hidden dimension d used throughout the retriever.

3.2.2. Structural Encoding

To capture the topological distance and directional proximity relative to the topic entities S_q, we compute a Directional Distance Encoding (DDE) [5], denoted as x_e^{DDE}, for each entity e in $\mathcal{G}_q$. Concretely, DDE performs bidirectional feature diffusion on the directed candidate subgraph $\mathcal{G}_q$, initialized with a topic-indicator signal.

We initialize a scalar signal $s_e^{(0)} \in \{0, 1\}$ such that $s_e^{(0)} = 1$ if $e \in S_q$ and 0 otherwise. For diffusion step $k = 0, \ldots, L_{\text{DDE}} - 1$, we propagate along incoming edges:

$$s_e^{(k+1)} = \text{MEAN}\{s_{e'}^{(k)} \mid (e', \cdot, e) \in \mathcal{G}_q\}, \tag{3}$$

and we additionally propagate along the reverse direction to account for the directed nature of KG triples:

$$s_e^{(r,0)} = s_e^{(0)}, \quad s_e^{(r,k+1)} = \text{MEAN}\{s_{e'}^{(r,k)} \mid (e, \cdot, e') \in \mathcal{G}_q\}. \tag{4}$$

Finally, we concatenate the forward and reverse diffusion states to obtain the DDE feature:

$$x_e^{DDE} = [s_e^{(0)} \parallel s_e^{(1)} \parallel \cdots \parallel s_e^{(L_{\text{DDE}})} \parallel s_e^{(r,1)} \parallel \cdots \parallel s_e^{(r,L_{\text{DDE}})}]. \tag{5}$$

L_{DDE} is treated as a small constant hyperparameter controlling the diffusion radius. In our experiments, we set $L_{\text{DDE}} = 3$. For a node with an empty neighborhood, we define the mean over the empty set to be 0.

3.2.3. Initial Entity Hidden States

The initial hidden state for each entity e is constructed by concatenating its semantic and structural features, followed by a projection:

$$h_e^{(0)} = \text{MLP}_{\text{init}}([\tilde{z}_e \parallel x_e^{DDE}]). \tag{6}$$

Relation embeddings $\tilde{z}_r$ and the query embedding $\tilde{z}_q$ are reused across all layers for edge gating and final triple scoring.

3.3. Coarse-to-Fine Retrieval Pipeline

To balance the efficiency of lightweight triple retrieval with the necessity of enhancing evidence quality under a strict retrieval budget, we implement a coarse-to-fine retrieval pipeline. This pipeline sequentially refines the set of candidate triples, first generating a broad set of candidates and then re-ranking them with a more sophisticated model.

3.3.1. Coarse Retrieval

Initially, a lightweight DDE+MLP retriever is applied to the question-specific candidate subgraph $\mathcal{G}_q$ to generate $\mathcal{C}_q$, a compact set of high-recall candidate triples:

$$\mathcal{C}_q = \text{TopK}_{K_c}(\mathcal{G}_q;\ s_{\text{coarse}}), \tag{7}$$

where K_c represents the budget for coarse retrieval.

To enable fast, parallel scoring over $\mathcal{G}_q$, we implement $s_{\text{coarse}}(\tau)$ as a lightweight binary relevance estimator that integrates semantic embeddings (Section 3.2.1) and DDE structural features (Section 3.2.2). For a triple $\tau = (h, r, t)$, we first form a triple-level DDE representation by concatenating the endpoint encodings:

$$z_\tau^{DDE} = [x_h^{DDE} \parallel x_t^{DDE}]. \tag{8}$$

The coarse relevance score is then computed as:

$$s_{\text{coarse}}(\tau) = \sigma\left(\text{MLP}_{\text{coarse}}([\tilde{z}_q \parallel \tilde{z}_h \parallel \tilde{z}_r \parallel \tilde{z}_t \parallel z_\tau^{DDE}])\right), \tag{9}$$

where $\tilde{z}_h$ and $\tilde{z}_t$ are the projected semantic embeddings of entities h and t, respectively, and $\sigma(\cdot)$ denotes the sigmoid function. The coarse scorer $\text{MLP}_{\text{coarse}}$ is parameterized independently of the fine-stage scorer $\text{MLP}_{\text{score}}$. During inference, we compute $s_{\text{coarse}}(\tau)$ for all $\tau \in \mathcal{G}_q$ in parallel and select the Top-K_c triples to form $\mathcal{C}_q$.

This stage leverages the robust recall performance of DDE-based retrieval, acting as an effective initial filtering mechanism.

3.3.2. Fine Retrieval

Following coarse retrieval, we form a compact triple subgraph by keeping exactly the candidate triple set, i.e., $\tilde{\mathcal{G}}_q = \mathcal{C}_q$, and denote its incident entity set as $\mathcal{V}(\tilde{\mathcal{G}}_q) = \mathcal{V}(\mathcal{C}_q)$. The proposed SPIMP-based GNN encoder is then exclusively applied to $\tilde{\mathcal{G}}_q$ to generate context-aware entity representations. Triple scores are subsequently recomputed for all $\tau \in \mathcal{C}_q$ using these SPIMP-enhanced states, and the final Top-K triples are selected as the refined retrieved evidence.

This design explicitly targets the low-K retrieval regime, which is particularly relevant given LLM context constraints. Rather than replacing the lightweight coarse retriever, SPIMP reallocates computational effort to a compact, high-recall candidate subgraph, thereby improving the

information density and structural consistency of the top-ranked triples. The synergistic operation of these two stages ensures both broad coverage and precise selection, optimizing for the constraints of downstream LLM processing.

3.4. Structure-Prior Injected Message Passing (SPIMP)

Conventional GNNs often aggregate neighborhood information uniformly or based solely on semantic relevance, which can lead to isotropic noise diffusion within dense candidate subgraphs. To mitigate this, we introduce Structure-Prior Injected Message Passing (SPIMP). In SPIMP, DDE is leveraged as a structural prior to modulate the information flow along each edge. For a directed triple $\tau = (h, r, t)$, the message transmitted from entity h to entity t at layer l is controlled by a topology-aware coefficient $\alpha^{(l)}_{h,r,t}$:

$$\alpha^{(l)}_{h,r,t} = \sigma\left(\mathrm{MLP}_{\mathrm{gate}}([h^{(l-1)}_h \parallel \tilde{z}_r \parallel \tilde{z}_q \parallel x^{DDE}_h \parallel x^{DDE}_t])\right). \tag{10}$$

We use a vector-valued gate $\alpha^{(l)}_{h,r,t} \in \mathbb{R}^d$ and apply $\sigma(\cdot)$ element-wise, such that each dimension is independently bounded in $(0, 1)$. Intuitively, $\alpha^{(l)}_{h,r,t}$ performs dimension-wise modulation of information flow along the directed triple (h, r, t) by jointly considering semantic compatibility (from the question, relation, and source node state) and the structural orientation induced by DDE. The edge message is then computed as:

$$m^{(l)}_{h,r,t} = \alpha^{(l)}_{h,r,t} \odot \left(W_V h^{(l-1)}_h + \tilde{z}_r\right), \tag{11}$$

where W_V represents a learnable linear transformation and $\odot$ denotes the element-wise (Hadamard) product. For each target entity t, incoming messages are aggregated over its incoming triple set $\mathcal{I}(t) = \{(u, r, t) \in \tilde{\mathcal{G}}_q\}$:

$$h^{(l)}_{\mathcal{I}(t)} = \sum_{(u,r,t)\in\mathcal{I}(t)} m^{(l)}_{u,r,t}, \tag{12}$$

SPIMP achieves anisotropic message routing by jointly considering semantic relevance and topic-centric structural priors; Fig. 2 summarizes the per-layer message generation, aggregation, and adaptive fusion with residual bypass.

3.5. Adaptive Semantic-Structural Fusion (ASSF)

While DDE offers robust structural guidance, an excessive reliance on structural priors can compromise robustness, particularly in cases of KG incompleteness or entity-linking inaccuracies. To prevent rigid routing, we incorporate an adaptive residual bypass mechanism. This mechanism dynamically balances newly aggregated topology-aware messages with previous hidden states. For each entity t at layer l, a fusion gate is computed and the hidden state is updated accordingly:

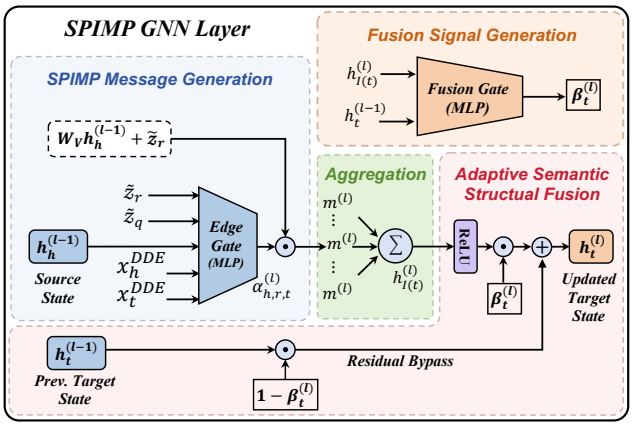

Figure 2: Detailed computation in one SPIMP GNN layer. The edge gate $\alpha^{(l)}_{h,r,t}$ injects DDE priors into relation-aware message generation; incoming messages are aggregated and fused with a residual bypass via the gate $\beta^{(l)}_t$ (ASSF) to produce the updated state $h^{(l)}_t$.

$$\beta^{(l)}_t = \sigma(W_\beta[h^{(l)}_{\mathcal{I}(t)} \parallel h^{(l-1)}_t]), \tag{13}$$

$$h^{(l)}_t = \beta^{(l)}_t \odot \mathrm{ReLU}(h^{(l)}_{\mathcal{I}(t)}) + (1 - \beta^{(l)}_t) \odot h^{(l-1)}_t, \tag{14}$$

This fusion mechanism allows the model to dynamically determine the extent to which it trusts the structure-guided neighborhood aggregation versus the entity's previous semantic-structural state.

3.6. Triple Scoring for Retrieval

After L layers of SPIMP, we obtain context-aware entity representations, denoted as $h^{(L)}_e$. Subsequently, each candidate triple $\tau = (h, r, t) \in \mathcal{C}_q$ is scored using the updated endpoint states in conjunction with the query and relation embeddings:

$$s(\tau) = \mathrm{MLP}_{\mathrm{score}}([h^{(L)}_h \parallel \tilde{z}_r \parallel h^{(L)}_t \parallel \tilde{z}_q]), \tag{15}$$

In this setup, the GNN functions as a backbone encoder: SPIMP contextualizes entities through structure-aware message passing, while $\mathrm{MLP}_{\mathrm{score}}$ performs the final query-aware triple ranking in parallel. During inference, all candidate triples within $\mathcal{C}_q$ are ranked according to their $s(\tau)$ scores. The Top-K triples are then selected and passed to the downstream LLM as the retrieved evidence subgraph.

3.7. Optimization with Confidence-Weighted Weak Supervision

The retriever is trained under weak supervision using question-answer pairs (q, A_q), circumventing the need for gold-annotated reasoning paths, where $A_q \subseteq \mathcal{E}$ denotes the set of gold answer entities for q. Throughout training, we

maintain consistency between training and inference by using the same fixed candidate-graph protocol to obtain $\mathcal{G}_q$ and applying the same coarse-to-fine pipeline to obtain C_q and $\tilde{\mathcal{G}}_q$. Since gold reasoning triples are typically unavailable, we construct confidence-weighted surrogate supervision by assigning each triple $\tau \in \mathcal{G}_q$ a soft target $w(\tau) \in [0, 1]$, and use it to optimize both the coarse and fine retrieval stages.

3.7.1. Confidence-Weighted Surrogate Evidence

Goal. For each candidate triple $\tau \in \mathcal{G}_q$, we construct a soft target $w(\tau) \in [0, 1]$ that reflects our confidence that τ provides useful evidence for connecting the linked topic entities S_q to the gold answer set A_q (available only during training).

Intuition. In an undirected view of the candidate graph, triples on topic-to-answer shortest paths are the most reliable positives; triples on slightly longer (near-shortest) paths are weaker positives. We additionally allow a small number of attribute/type triples around the involved entities as auxiliary evidence for downstream LLM reasoning.

We first form an undirected, unweighted graph G_q^{und} by treating each triple $\tau = (h, r, t) \in \mathcal{G}_q$ as an undirected edge between h and t. Let $\text{dist}_{G_q^{\text{und}}}(\cdot, \cdot)$ denote the shortest-path distance on G_q^{und}, with ∞ for unreachable pairs. We define

$$d_S(v) = \min_{e \in S_q} \text{dist}_{G_q^{\text{und}}}(e, v), \tag{16}$$

$$d_A(v) = \min_{a \in A_q} \text{dist}_{G_q^{\text{und}}}(a, v), \tag{17}$$

$$d^* = \min_{a \in A_q} d_S(a), \tag{18}$$

If $d^* = \infty$, we discard the corresponding training instance. For each triple $\tau = (h, r, t)$, we further define the path-length proxy

$$\ell(\tau) = \min\left(d_S(h) + 1 + d_A(t),\ d_S(t) + 1 + d_A(h)\right), \tag{19}$$

which estimates the length of the shortest topic-to-answer connection that uses τ as a bridging edge (the $+1$ accounts for traversing τ itself).

Algorithm 1 summarizes the construction of confidence-weighted targets using (i) shortest-path triples, (ii) a capped set of near-shortest triples selected with the fixed semantic prior $s_{\text{sem}}(\tau)$ (Eq. (1)), and (iii) auxiliary triples filtered by a relation whitelist $\mathcal{R}_{type}$ and a per-entity cap B_{type}.

Hyperparameters. In Algorithm 1, the hyperparameters $(\Delta, P_{near}, B_{type})$ control the size of the surrogate triple subsets and thus the trade-off between coverage and label noise. Specifically, Δ sets the slack for defining near-shortest triples, P_{near} caps the number of selected near-shortest triples, and B_{type} limits the number of auxiliary attribute triples retained per entity. The confidence weights (α, β) are fixed soft targets assigned to near-shortest and auxiliary attribute triples, respectively, reflecting their lower reliability compared to exact shortest-path triples (with $w(\tau) = 1$). We study the sensitivity of these hyperparameters in Section 5.2.1.

Algorithm 1 Construction of Confidence-Weighted Surrogate Targets

Require: Candidate subgraph $\mathcal{G}_q$, topic entities S_q, answer entities A_q, semantic score $s_{\text{sem}}(\tau)$, slack Δ, cap P_{near}, type whitelist $\mathcal{R}_{type}$, per-entity cap B_{type}, weights α, β
Ensure: Soft targets $\{w(\tau) \mid \tau \in \mathcal{G}_q\}$

```
Construct G_q^und from G_q
Compute d_S(v), d_A(v), and d* by Eqs. (16)–(18)
if d* = ∞ then
    discard this training instance
Initialize w(τ) ← 0 for all τ ∈ G_q
P_sp ← {τ ∈ G_q : ℓ(τ) = d*}
for all τ ∈ P_sp do
    w(τ) ← 1
P_near^raw ← {τ ∈ G_q : d* < ℓ(τ) ≤ d* + Δ}
P_near ← TopK_{P_near}(P_near^raw; s_sem)
for all τ ∈ P_near do
    w(τ) ← max(w(τ), α)
P_aux ← {τ = (e, r, c) ∈ G_q : r ∈ R_type, e ∈ V(P_sp) ∪ S_q ∪ A_q}
For each entity e, retain at most B_type triples in P_aux incident to e, ranked by s_sem(τ)
for all retained τ ∈ P_aux do
    w(τ) ← max(w(τ), β)
return {w(τ) | τ ∈ G_q}
```

Unless otherwise stated, we use $\Delta = 1$, $P_{near} = 500$, $B_{type} = 3$, $\alpha = 0.5$, and $\beta = 0.2$. In summary, we assign $w(\tau) = 1$ to shortest-path triples, $w(\tau) = \alpha$ to near-shortest triples, $w(\tau) = \beta$ to auxiliary type triples, and $w(\tau) = 0$ otherwise (Algorithm 1). Note that A_q is used only during training to construct $w(\tau)$; at inference time, retrieval conditions only on the question q, the linked topic entities, and the fixed candidate subgraph.

3.7.2. Two-Stage Training of Coarse and Fine Retrievers

The discrete $\text{TopK}(\cdot)$ operator used for forming C_q makes candidate selection non-differentiable. We therefore adopt a two-stage training protocol. We first pretrain the coarse retriever (the projection matrices W_q, W_e, W_r and $\text{MLP}_{\text{coarse}}$) using the confidence-weighted targets over $\mathcal{G}_q$:

$$\mathcal{L}_{\text{coarse}} = -\sum_{\tau \in \mathcal{G}_q} \left(w(\tau) \log s_{\text{coarse}}(\tau) + (1 - w(\tau)) \log(1 - s_{\text{coarse}}(\tau))\right). \tag{20}$$

After pretraining, we freeze the coarse retriever and use it to construct $C_q = \text{TopK}_{K_c}(\mathcal{G}_q;\ s_{\text{coarse}})$. We do not backpropagate through the Top-K candidate selection.

3.7.3. Weighted Training Objective

We interpret the fine-stage score $s(\tau)$ as a logit and compute the relevance probability as $p(\tau) = \sigma(s(\tau))$. We

then optimize the retriever with a soft-label Binary Cross-Entropy objective over the candidate triples in C_q:

$$\mathcal{L}_{\mathrm{WBCE}} = -\sum_{\tau \in C_q} \left(w(\tau) \log p(\tau) + (1 - w(\tau)) \log(1 - p(\tau)) \right). \tag{21}$$

This confidence-weighted objective reduces the impact of noisy surrogate supervision while preserving the ability to rank triples under a fixed Top-K retrieval budget. In the fine stage, we optimize the SPIMP-based retriever with $\mathcal{L}_{\mathrm{WBCE}}$ over $\mathcal{C}_q$.

4. EXPERIMENTS

This section describes the experimental protocol for evaluating our coarse-to-fine triple retriever and its impact on downstream knowledge graph question answering (KGQA). We evaluate (i) Top-K triple retrieval quality under varying budgets and (ii) end-to-end KGQA performance with a fixed LLM reasoner.

4.1. Experimental Setup

4.1.1. Datasets

We conduct experiments on two widely used Freebase-based [10] KGQA benchmarks: WebQSP [11] and ComplexWebQuestions (CWQ) [54]. WebQSP contains 4,737 natural language questions that are answerable using a subset Freebase KG. CWQ contains 34,699 total complex questions that require up to 4-hops of reasoning over the KG. We use the standard train/dev/test splits provided by each benchmark.

Filtered evaluation. Following prior work, we additionally report results on a filtered subset (denoted with the suffix "-sub") that removes questions whose gold answers are not present in the underlying KG. This filtering prevents retrieval metrics from being penalized by KG incompleteness and enables a more faithful assessment of evidence retrieval.

4.1.2. Topic Entities and Candidate Graph Protocol

For each question q, we obtain the topic entity set $\mathcal{S}_q$ from an entity-linking preprocessing pipeline and treat it as a fixed input for all methods. Unless otherwise noted, all retrievers operate on the same question-centered rough candidate subgraph $\mathcal{G}_q$.

Rough subgraphs. To ensure fair comparison with strong baselines, we follow the evaluation convention of SubgraphRAG and reuse the released topic entities and rough subgraphs provided by prior work [2, 5]. We denote the resulting question-specific triple set uniformly as $\mathcal{G}_q$. Since the candidate-space construction relies on discrete selection, we treat $\mathcal{G}_q$ as a fixed preprocessing artifact and do not backpropagate through it.

4.1.3. Retrieval Budgets

Our retriever uses a coarse-to-fine pipeline with a coarse budget K_c and a final budget K. Unless stated otherwise, we set $K_c = 600$ and use $K = 100$ by default for downstream LLM reasoning. For retrieval-curve evaluation over the final evidence budget, we follow Fig. 3. On WebQSP, we evaluate K at 5, 15, 25, 50, 100, 200, and 400. On CWQ, we evaluate K at 20, 50, 100, 200, and 400. Sensitivity to the coarse budget K_c is reported in Fig. 4.

4.1.4. Baselines

We group baselines by whether they directly produce a ranked list of triples under a fixed Top-K budget.

Triple retrievers. Under the same candidate-graph protocol, we compare seven triple retrievers: Cosine similarity [26], MLP, MLP + topic entity, GraphSAGE [27], GraphSAGE + topic entity, the SubgraphRAG retriever [5], and our SPIMP-RAG. All methods output a ranked list on $\mathcal{G}_q$, from which we take Top-K triples for evaluation.

End-to-end KGQA systems. Table 1 includes LLM-only and prompting baselines such as Flan-T5-xl [55], Alpaca-7B [56], and KD+CoT[57]. It also includes ChatGPT, and ToG [1]. The KG+LLM baselines include RoG[2] and G-Retriever[53], as well as GNN-RAG [3] and SubgraphRAG [5]. We report our SPIMP-RAG under the same table. For non-ours methods, we follow the original settings indicated in Table 1. We use released checkpoints when available and otherwise report official published numbers.

4.1.5. Evaluation Metrics

Retrieval metrics. For Top-K triple retrieval, we compute per-question metrics and then average across the dataset: (1) *Answer Entity Recall@K*, the fraction of gold answer entities that appear in the incident entity set of the retrieved triples; (2) *Shortest-Path Triple Recall@K*, where positives are triples on topic-to-answer shortest paths within the undirected view G_q^{und} (i.e., $\mathcal{P}_{\mathrm{sp}}$ in Section 3); (3) *GPT-4o Triple Recall@K*, where positives are a set of question-specific supporting triples labeled by GPT-4o following prior work [5], and the metric measures the fraction of these labeled triples covered by the retrieved Top-K set; and (4) *Path Coverage@K*, which measures whether the retrieved triples contain at least one topic-to-answer connecting chain. Concretely, let $\mathcal{R}_q^{(K)}$ be the retrieved Top-K triples and let $G(\mathcal{R}_q^{(K)})$ be the undirected graph obtained by treating each triple $(h, r, t) \in \mathcal{R}_q^{(K)}$ as an undirected edge between h and t; Path Coverage@K is 1 if there exist $e \in \mathcal{S}_q$ and $a \in A_q$ such that $\mathrm{dist}_{G(\mathcal{R}_q^{(K)})}(e, a) < \infty$, and 0 otherwise.

End-to-end KGQA metrics. For downstream question answering, we report F1 and Hit@1. Here Hit@1 is a set-level metric that measures whether at least one gold answer is included in the predicted answer set.

4.1.6. Efficiency

We measure wall-clock latency for candidate-independent retriever stages and exclude external KG query time. For methods that use text embeddings, we only include the time for question encoding since entity and relation embeddings can be cached. All experiments are conducted on a machine with 4×RTX 4090 GPUs and 128GB RAM.

4.2. Implementation Details

4.2.1. Text Encoder and Surface Forms

We use a frozen dense text encoder, BAAI/bge-large-en-v1.5, to produce semantic embeddings z_q, z_e, and z_r for questions, entities, and relations, respectively (Section 3.2.1). Entity and relation embeddings are precomputed offline; at inference time, only the question embedding is computed online.

To construct textual inputs for the encoder, we use human-readable surface forms: entities are represented by their canonical names (and aliases when available), and relations are represented by normalized relation names obtained from their KG identifiers (e.g., replacing separators with spaces).

4.2.2. Retriever Configuration

Unless stated otherwise, we set the hidden dimension to $d = 256$ and use $L = 2$ SPIMP layers. We apply dropout of 0.1 to MLP modules. For DDE, we compute x_e^{DDE} via bidirectional diffusion on $\mathcal{G}_q$ (Section 3.2.2) with $L_{\mathrm{DDE}} = 3$.

4.2.3. Training Details

We adopt the two-stage training protocol described in Section 3. We first pretrain the coarse retriever (projection matrices and $\mathrm{MLP}_{\mathrm{coarse}}$) on $\mathcal{G}_q$ using the confidence-weighted objective $\mathcal{L}_{\mathrm{coarse}}$, freeze it, and construct $\mathcal{C}_q$ via Top-K_c selection. We then train the fine-stage SPIMP retriever on $\mathcal{C}_q$ using $\mathcal{L}_{\mathrm{WBCE}}$. We optimize with AdamW (learning rate 1×10^{-4}, weight decay 1×10^{-2}) and select hyperparameters on the development set with early stopping based on retrieval performance.

4.2.4. LLM Reasoners and Prompting

Unless stated otherwise, we evaluate end-to-end KGQA using Llama-3.1-8B-Instruct as the default reasoner without finetuning. Retrieved triples are linearized into textual facts of the form "head [relation] tail" and concatenated with the question using a fixed prompt template. We use deterministic decoding with temperature 0.

5. RESULTS

This section reports comparative evaluation, sensitivity analysis, and ablation studies of our coarse-to-fine retriever. To keep the presentation concise, we report key operating points in the main text and provide full curves and additional diagnostics in the appendix.

5.1. Comparative Evaluation

5.1.1. Retrieval Results

We evaluate low-budget triple retrieval quality under the same candidate graphs $\mathcal{G}_q$. In our pipeline, the final evidence is defined as the Top-K triples after the fine retrieval stage (SPIMP reranking) over the coarse candidate set. All baselines output a ranked list of triples under the same candidate-graph protocol, from which we take the Top-K for evaluation. Fig. 3 reports retrieval trends across budgets for the seven compared retrievers using Shortest-Path Triple Recall@K, GPT-4o Triple Recall@K, and Answer Entity Recall@K.

Table 1
End-to-end KGQA performance on WebQSP and CWQ. We report Hit@1 (%) and F1 (%), where Hit@1 measures whether at least one gold answer is included in the predicted answer set. Our method uses Llama-3.1-8B-Instruct as the default reasoner; other methods follow their original settings/backbones as indicated by their names.

Methods	WebQSP		CWQ	
	Hit@1	F1	Hit@1	F1
GraftNet [58]	66.7	62.4	36.8	32.7
PullNet [59]	68.1	-	45.9	-
NSM [17]	68.7	62.8	47.6	42.4
UniKGQA [39]	77.2	72.2	51.2	49.1
ReaRev [18]	76.4	70.9	52.9	47.8
Flan-T5-xl [55]	31.0	-	14.7	-
Alpaca-7B [56]	51.8	-	27.4	-
ChatGPT [3]	66.8	-	39.9	-
ChatGPT+CoT [3]	75.6	-	48.9	-
KD+CoT [57]	68.6	52.5	55.7	-
ToG+ChatGPT [1]	76.2	-	58.9	-
G-Retriever [53]	70.1	-	-	-
SubgraphRAG+Llama3.1-8B [5]	86.61	70.57	56.98	47.16
SPIMP-RAG + Llama-3.1-8B	**88.13**	**72.93**	**59.43**	**51.82**

5.1.2. End-to-End KGQA Results

Table 1 summarizes end-to-end KGQA performance on WebQSP and CWQ. Unless stated otherwise, our method uses Llama-3.1-8B-Instruct as the default reasoner and the Top-100 retrieved triples as evidence. For other systems, we report their performance under the original settings/backbones as indicated by their names.

5.2. Sensitivity Analysis

Retrieval Budgets (K and K_c). We analyze sensitivity to both the final budget K and the coarse budget K_c. Fig. 3 reports retrieval trends across final evidence budgets K. Fig. 4 shows coarse-stage recall as a function of K_c on WebQSP and CWQ using Answer Entity Recall, Shortest-Path Triple Recall, and GPT-4o Triple Recall. We observe clear diminishing returns beyond moderate coarse budgets, so we set $K_c = 600$ as the default to balance candidate coverage and computation. Full budget curves are reported in Appendix D.

5.2.1. Sensitivity to Surrogate-Target Hyperparameters

We analyze robustness of the confidence-weighted surrogate targets by sweeping the hyperparameters in Algorithm 1 while keeping the candidate-graph protocol and retriever architecture fixed. We consider $\Delta \in \{0, 1, 2\}$, $P_{near} \in \{100, 500, 1000\}$, $B_{type} \in \{1, 3, 5\}$, $\alpha \in \{0.3, 0.5, 0.7\}$, and $\beta \in \{0.1, 0.2, 0.3\}$. For each panel in Fig. 5, we jointly vary the hyperparameters shown on that panel while fixing the remaining settings at their default values. We report retrieval performance at $K = 20$ (Answer Entity Recall@K and Path Coverage@K) and end-to-end KGQA performance at the

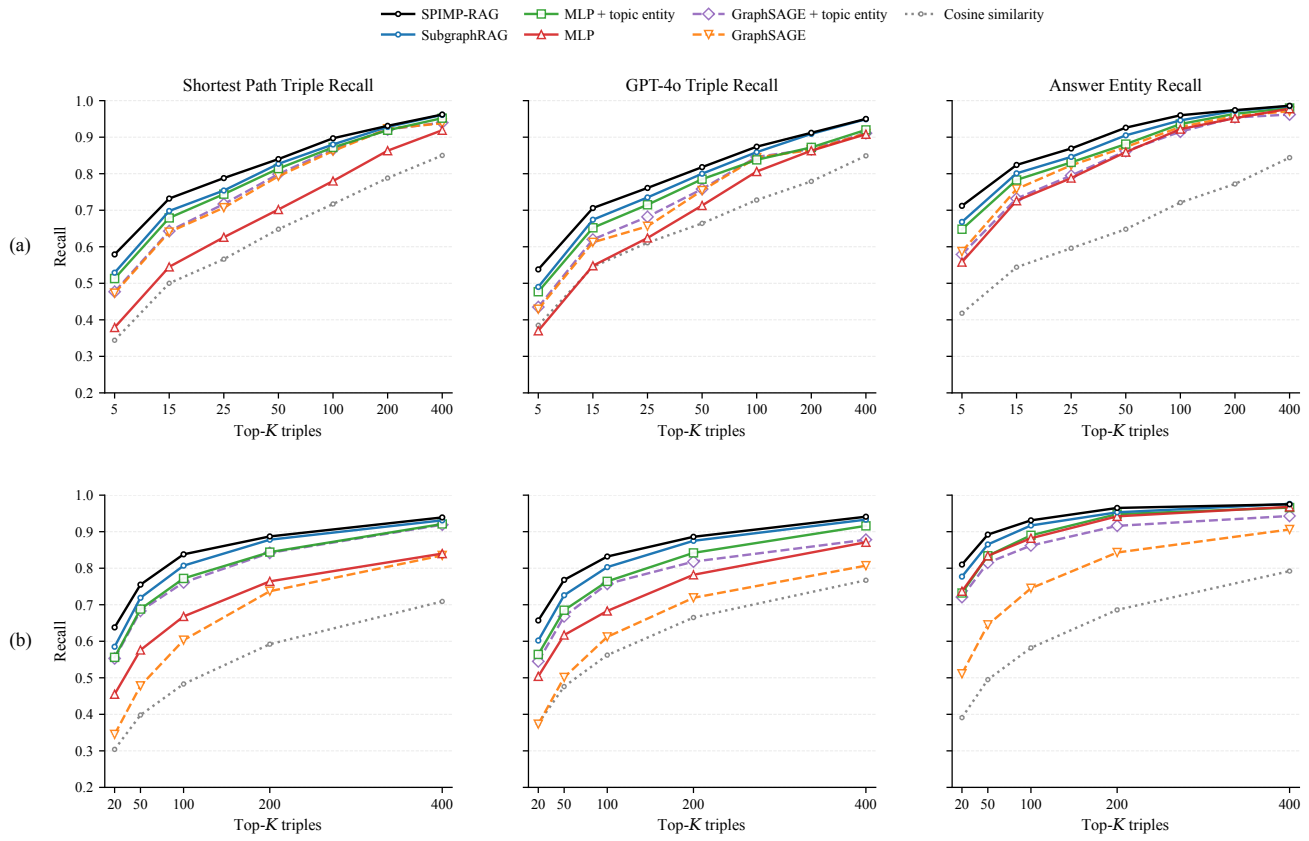

Figure 3: Retrieval results under the same candidate-graph protocol across evidence budgets K. The top row (a) reports WebQSP and the bottom row (b) reports CWQ; columns show Shortest-Path Triple Recall@K, GPT-4o Triple Recall@K, and Answer Entity Recall@K, respectively.

default evidence budget $K = 100$ (Hit@1 and F1). Fig. 5 visualizes representative views of the three hyperparameter groups; Hit@1 follows the same overall trend as F1 and is omitted from the figure for readability.

Overall, the confidence-weighted surrogate targets are reasonably robust around the default setting, and the dominant pattern is a low-budget trade-off between structural coverage and label noise. The two datasets nevertheless prefer different operating points. WebQSP contains mostly short and topologically simple questions, so it benefits from concentrated supervision that suppresses noisy detours. CWQ contains longer and more compositional queries, so it benefits more from moderate structural relaxation and a small amount of auxiliary type evidence. This contrast is consistent with the different graph complexity of the two benchmarks.

Topology relaxation (Δ and P_{near}). The left column of Fig. 5 shows that WebQSP is only mildly helped by relaxing the shortest-path criterion from $\Delta = 0$ to $\Delta = 1$, while a larger slack $\Delta = 2$ quickly hurts AER@20 and Path Coverage@20, especially when P_{near} is also large. Under the strict $K = 20$ retrieval budget, overly permissive near-shortest supervision allows noisy detours to displace the key bridging triples. In contrast, CWQ benefits substantially from moving to $\Delta = 1$, which is expected for longer multi-hop graphs where exact shortest paths are more brittle. Increasing P_{near} from 500 to 1000 then slightly flattens or even reduces AER@20, yet F1 remains stable or slightly improves, suggesting that some additional near-shortest evidence is still useful for downstream reasoning even when low-budget retrieval becomes more diffuse.

Auxiliary type capacity (B_{type}). The right column shows that B_{type} is the clearest source of metric divergence. On WebQSP, increasing B_{type} steadily lowers Path Coverage@20 and mildly hurts end-to-end QA, indicating that simple questions require only a small amount of side information before attribute/type triples begin to crowd out the main reasoning chain. On CWQ, however, increasing B_{type} from 1 to 3 causes only a slight drop in Path Coverage@20 while improving Hit@1 and F1, which suggests that a few type constraints provide valuable semantic disambiguation for complex compositional questions. Once B_{type} reaches 5, the low-budget evidence set is again over-consumed and both retrieval and QA metrics deteriorate. This behavior also matches the ablation in Table 2, where removing $\mathcal{P}_{\text{aux}}$ hurts end-to-end quality despite smaller retrieval changes.

Confidence weights (α and β). The middle column further shows that moderate soft-label weights work best.

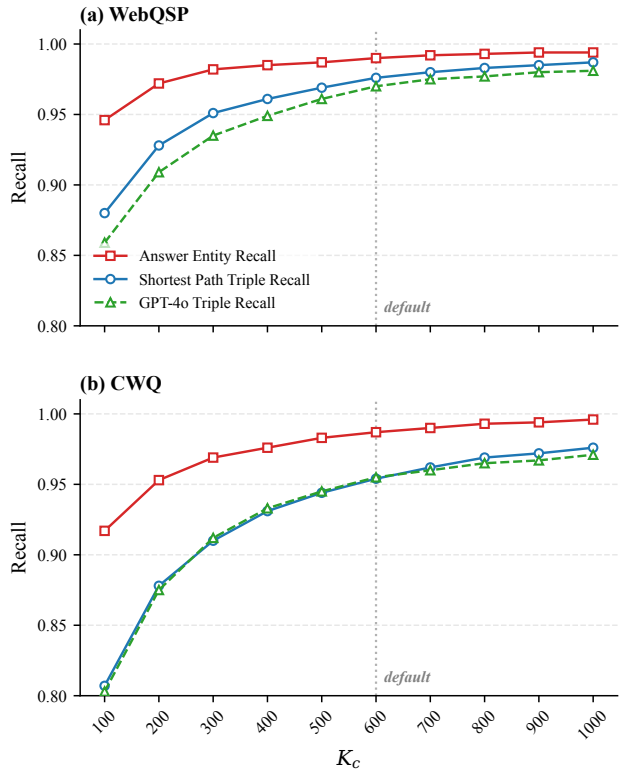

Figure 4: Sensitivity to the coarse budget K_c in the coarse retrieval stage. We report Answer Entity Recall, Shortest-Path Triple Recall, and GPT-4o Triple Recall on WebQSP (left) and CWQ (right). Performance saturates beyond moderate K_c, supporting the default setting $K_c = 600$.

On WebQSP, $\alpha = 0.5$ consistently outperforms $\alpha = 0.7$, indicating that over-weighting near-shortest triples blurs the distinction between exact bridge triples and weaker alternatives. On CWQ, the most sensitive parameter is β: setting $\beta = 0.3$ noticeably degrades F1, and the corresponding coverage curves show the sharpest drop in Path Coverage@20 because the retriever is encouraged to over-attend to leaf-like attribute triples instead of preserving multi-hop connectors. Taken together, these trends support the default cross-dataset setting $\Delta = 1$, $P_{near} = 500$, $B_{type} = 3$, $\alpha = 0.5$, and $\beta = 0.2$ as a robust compromise for the remaining experiments.

5.3. Ablation Study

We conduct ablations to isolate the contribution of key design choices in SPIMP-RAG under a fixed protocol: same topic entities and candidate subgraphs $\mathcal{G}_q$, same coarse budget K_c=600, same end-to-end evidence budget K=100, and the same training/LLM settings as the full model. We report retrieval quality with *Answer Entity Recall@50 (AER@50)*, which provides a stable moderate low-budget operating point already covered by the budget settings in Fig. 3, together with end-to-end Hit@1 (%) and F1 (%) at K=100. This choice stabilizes module-level comparisons and does not replace the strict low-K analysis from full budget curves.

Methods. (1) *w/o Fine Retrieval*: remove the SPIMP reranking stage and use coarse scores directly; (2) *w/o ASSF*: replace adaptive semantic-structural fusion with a standard residual update while keeping other SPIMP components unchanged; (3) *Hard labels*: binarize surrogate targets by setting all triples with $w(\tau)>0$ to 1; and (4) *w/o* $\mathcal{P}_{aux}$: remove auxiliary attribute triples from surrogate-target construction.

As shown in Table 2, removing fine retrieval yields the largest degradation, especially on end-to-end KGQA, indicating that SPIMP reranking is the main driver of high-quality low-budget evidence. Replacing ASSF or removing confidence weighting (hard labels) causes consistent medium drops, suggesting both robust fusion and soft weak supervision are important under noisy surrogate signals. Removing $\mathcal{P}_{\text{aux}}$ leads to relatively smaller retrieval changes but clearer end-to-end losses, which is consistent with the role of auxiliary type/attribute evidence in supporting final reasoning quality.

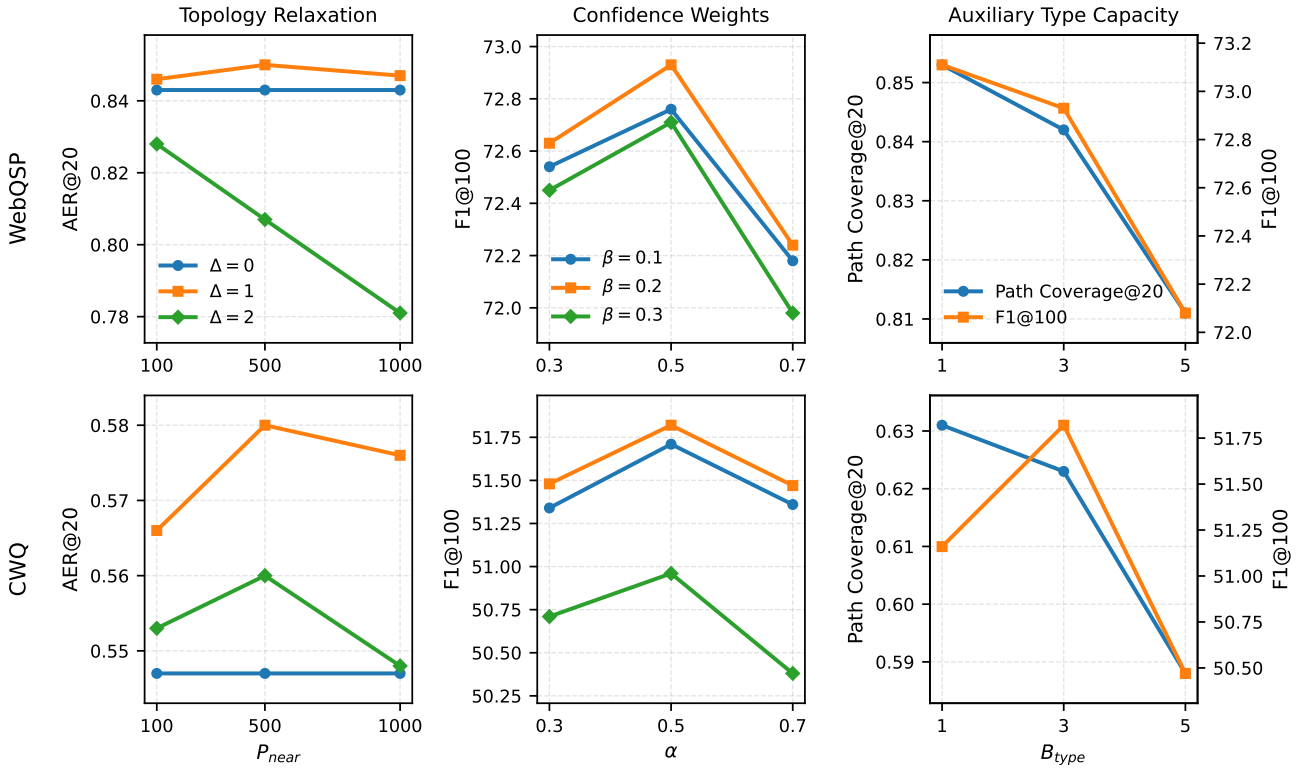

Figure 5: Sensitivity to surrogate-target hyperparameters. Left column: Answer Entity Recall@20 under joint sweeps of the topology-relaxation parameters (x-axis: P_{near}; lines: Δ). Middle column: F1 at $K = 100$ under joint sweeps of the confidence weights (x-axis: α; lines: β). Right column: Path Coverage@20 and F1 at $K = 100$ versus the auxiliary type budget B_{type}. Top row: WebQSP. Bottom row: CWQ. Across both datasets, moderate relaxation and confidence weighting provide the best trade-off between coverage and noise.

Table 2
Ablation study of our retriever. Retrieval quality is measured by Answer Entity Recall@50 (AER@50), and end-to-end performance is measured by Hit@1 (%) and F1 (%).

Method	WebQSP			CWQ		
	AER@50	Hit@1	F1	AER@50	Hit@1	F1
SPIMP-RAG	92.6	88.13	72.93	89.2	59.43	51.82
SPIMP-RAG w/o Fine Retrieval	90.3	86.21	70.13	85.3	56.61	47.05
SPIMP-RAG w/o ASSF	91.6	87.26	71.46	87.8	57.84	49.31
SPIMP-RAG w/o $\mathcal{P}_{\text{aux}}$	92.2	87.41	71.63	88.6	58.23	49.92
SPIMP-RAG w/ Hard labels	91.3	87.04	71.18	87.4	57.52	48.86

6. CONCLUSION

We presented a coarse-to-fine triple retrieval framework for knowledge graph question answering (KGQA) that targets the low-K regime required by LLM reasoners under strict context budgets. Our retriever combines a lightweight DDE+MLP coarse scorer for high-recall candidate generation with a structure-prior injected GNN reranker (SPIMP) and adaptive semantic-structural fusion (ASSF) for improved evidence ranking. To enable training without gold reasoning paths, we introduced a confidence-weighted surrogate supervision scheme and a two-stage optimization protocol that avoids backpropagation through discrete Top-K selection.

In future work, we will explore tighter coupling between retrieval and downstream generation, robustness to entity-linking errors and KG incompleteness, and more reliable supervision signals for training triple-level retrievers.

CRediT authorship contribution statement

Jun Chen: Methodology, Writing original draft, Funding acquisition. **Zhi-jun Xie:** Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research work is supported by the Key Technology Breakthrough Plan of Ningbo City's Science and Technology Innovation Yongjiang 2035 (Grant No. 2024Z166), and the Zhejiang Province Leading Earth Goose Program (Grant No.2025C02254(SD2)).

Data availability

The authors do not have permission to share data.

References

[1] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M Ni, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. *arXiv preprint arXiv:2307.07697*, 2023.

[2] Linhao Luo, Yuan-Fang Li, Gholamreza Haffari, and Shirui Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. *arXiv preprint arXiv:2310.01061*, 2023.

[3] Costas Mavromatis and George Karypis. Gnn-rag: Graph neural retrieval for efficient large language model reasoning on knowledge graphs. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 16682–16699, 2025.

[4] Guangyi Liu, Yongqi Zhang, Yong Li, and Quanming Yao. Dual reasoning: A gnn-llm collaborative framework for knowledge graph question answering. *arXiv preprint arXiv:2406.01145*, 2024.

[5] Mufei Li, Siqi Miao, and Pan Li. Simple is effective: The roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation. *arXiv preprint arXiv:2410.20724*, 2024.

[6] Yunshi Lan, Gaole He, Jinhao Jiang, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Complex knowledge base question answering: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(11):11196–11215, 2022.

[7] Bowen Jin, Gang Liu, Chi Han, Meng Jiang, Heng Ji, and Jiawei Han. Large language models on graphs: A comprehensive survey. *IEEE Transactions on Knowledge and Data Engineering*, 36(12):8622–8642, 2024.

[8] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3580–3599, 2024.

[9] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. Graph retrieval-augmented generation: A survey. *ACM Transactions on Information Systems*, 44(2):1–52, 2025.

[10] Kurt Bollacker, Colin Evans, Praveen Paritosh, Tim Sturge, and Jamie Taylor. Freebase: a collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD international conference on Management of data*, pages 1247–1250, 2008.

[11] Wen-tau Yih, Matthew Richardson, Christopher Meek, Ming-Wei Chang, and Jina Suh. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 201–206, 2016.

[12] Wen-tau Yih, Ming-Wei Chang, Xiaodong He, and Jianfeng Gao. Semantic parsing via staged query graph generation: Question answering with knowledge base. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1321–1331, 2015.

[13] Yunshi Lan and Jing Jiang. Query graph generation for answering multi-hop complex questions from knowledge bases. In *Proceedings of the 58th annual meeting of the association for computational linguistics*, pages 969–974, 2020.

[14] Yawei Sun, Lingling Zhang, Gong Cheng, and Yuzhong Qu. Sparqa: skeleton-based semantic parsing for complex questions over knowledge bases. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 8952–8959, 2020.

[15] Donghan Yu, Sheng Zhang, Patrick Ng, Henghui Zhu, Alexander Hanbo Li, Jun Wang, Yiqun Hu, William Wang, Zhiguo Wang, and Bing Xiang. Decaf: Joint decoding of answers and logical forms for question answering over knowledge bases. *arXiv preprint arXiv:2210.00063*, 2022.

[16] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, Cuiping Li, and Hong Chen. Subgraph retrieval enhanced model for multi-hop knowledge base question answering. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 5773–5784, 2022.

[17] Gaole He, Yunshi Lan, Jing Jiang, Wayne Xin Zhao, and Ji-Rong Wen. Improving multi-hop knowledge base question answering by learning intermediate supervision signals. In *Proceedings of the 14th ACM international conference on web search and data mining*, pages 553–561, 2021.

[18] Costas Mavromatis and George Karypis. Rearev: Adaptive reasoning for question answering over knowledge graphs. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 2447–2458, 2022.

[19] Yuyu Zhang, Hanjun Dai, Zornitsa Kozareva, Alexander Smola, and Le Song. Variational reasoning for question answering with knowledge graph. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[20] Mattia Atzeni, Jasmina Bogojeska, and Andreas Loukas. Sqaler: Scaling question answering by decoupling multi-hop and logical reasoning. *Advances in Neural Information Processing Systems*, 34:12587–12599, 2021.

[21] Xi Ye, Semih Yavuz, Kazuma Hashimoto, Yingbo Zhou, and Caiming Xiong. Rng-kbqa: Generation augmented iterative ranking for knowledge base question answering. In *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 6032–6043, 2022.

[22] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

[23] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.

[24] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. Modeling relational data with graph convolutional networks. In *European semantic web conference*, pages 593–607. Springer, 2018.

[25] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. Qa-gnn: Reasoning with language models and knowledge graphs for question answering. In *Proceedings of the 2021 conference of the North American chapter of the association for computational linguistics: human language technologies*, pages 535–546, 2021.

[26] Shiyang Li, Yifan Gao, Haoming Jiang, Qingyu Yin, Zheng Li, Xifeng Yan, Chao Zhang, and Bing Yin. Graph reasoning for question answering with triplet retrieval. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 3366–3375, 2023.

[27] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[28] Pan Li, Yanbang Wang, Hongwei Wang, and Jure Leskovec. Distance encoding: Design provably more powerful neural networks for graph representation learning. *Advances in Neural Information Processing Systems*, 33:4465–4478, 2020.

[29] Muhan Zhang, Pan Li, Yinglong Xia, Kai Wang, and Long Jin. Labeling trick: A theory of using graph neural networks for multi-node representation learning. *Advances in Neural Information Processing Systems*, 34:9061–9073, 2021.

[30] Hyeong Kyu Choi, Seunghun Lee, Jaewon Chu, and Hyunwoo J Kim. Nutrea: Neural tree search for context-guided multi-hop kgqa. *Advances in Neural Information Processing Systems*, 36:35954–35965, 2023.

[31] Zijian Li, Qingyan Guo, Jiawei Shao, Lei Song, Jiang Bian, Jun Zhang, and Rui Wang. Graph neural network enhanced retrieval for question answering of large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6612–6633, 2025.

[32] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, Cehao Yang, Jiaxin Mao, and Jian Guo. Think-on-graph 2.0: Deep and faithful large language model reasoning with knowledge-guided retrieval augmented generation. *arXiv preprint arXiv:2407.10805*, 2024.

[33] Runxuan Liu, Luobei Luobei, Jiaqi Li, Baoxin Wang, Ming Liu, Dayong Wu, Shijin Wang, and Bing Qin. Ontology-guided reverse thinking makes large language models stronger on knowledge graph question answering. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15269–15284, 2025.

[34] Dhaval Taunk, Lakshya Khanna, Siri Venkata Pavan Kumar Kandru, Vasudeva Varma, Charu Sharma, and Makarand Tapaswi. Grapeqa: Graph augmentation and pruning to enhance question-answering. In *Companion Proceedings of the ACM Web Conference 2023*, pages 1138–1144, 2023.

[35] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[36] Bowen Jin, Chulin Xie, Jiawei Zhang, Kashob Kumar Roy, Yu Zhang, Zheng Li, Ruirui Li, Xianfeng Tang, Suhang Wang, Yu Meng, et al. Graph chain-of-thought: Augmenting large language models by reasoning on graphs. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 163–184, 2024.

[37] Ruilin Zhao, Feng Zhao, Long Wang, Xianzhi Wang, and Guandong Xu. Kg-cot: Chain-of-thought prompting of large language models over knowledge graphs for knowledge-aware question answering. In *IJCAI*, pages 6642–6650, 2024.

[38] Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. Structgpt: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 9237–9251, 2023.

[39] Jinhao Jiang, Kun Zhou, Wayne Xin Zhao, and Ji-Rong Wen. Unikgqa: Unified retrieval and reasoning for solving multi-hop question answering over knowledge graph. *arXiv preprint arXiv:2212.00959*, 2022.

[40] Jinheon Baek, Alham Fikri Aji, and Amir Saffari. Knowledge-augmented language model prompting for zero-shot knowledge graph question answering. In *Proceedings of the 1st Workshop on Natural Language Reasoning and Structured Explanations (NLRSE)*, pages 78–106, 2023.

[41] Tianle Li, Xueguang Ma, Alex Zhuang, Yu Gu, Yu Su, and Wenhu Chen. Few-shot in-context learning on knowledge base question answering. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6966–6980, 2023.

[42] Yiming Tan, Dehai Min, Yu Li, Wenbo Li, Nan Hu, Yongrui Chen, and Guilin Qi. Can chatgpt replace traditional kbqa models? an in-depth analysis of the question answering performance of the gpt llm family. In *International Semantic Web Conference*, pages 348–367. Springer, 2023.

[43] Guanming Xiong, Junwei Bao, and Wen Zhao. Interactive-kbqa: Multi-turn interactions for knowledge base question answering with large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 10561–10582, 2024.

[44] Haoran Luo, E Haihong, Zichen Tang, Shiyao Peng, Yikai Guo, Wentai Zhang, Chenghao Ma, Guanting Dong, Meina Song, Wei Lin, et al. Chatkbqa: A generate-then-retrieve framework for knowledge base question answering with fine-tuned large language models. In *Findings of the association for computational linguistics: ACL 2024*, pages 2039–2056, 2024.

[45] Yike Wu, Nan Hu, Sheng Bi, Guilin Qi, Jie Ren, Anhuan Xie, and Wei Song. Retrieve-rewrite-answer: A kg-to-text enhanced llms framework for knowledge graph question answering. *arXiv preprint arXiv:2309.11206*, 2023.

[46] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

[47] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.

[48] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *The Twelfth International Conference on Learning Representations*, 2023.

[49] Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Richard James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, et al. Ra-dit: Retrieval-augmented dual instruction tuning. In *The Twelfth International Conference on Learning Representations*, 2023.

[50] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How

language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.

[51] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[52] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, and Liang Zhao. Grag: Graph retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 4145–4157, 2025.

[53] Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. *Advances in Neural Information Processing Systems*, 37:132876–132907, 2024.

[54] Alon Talmor and Jonathan Berant. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pages 641–651, 2018.

[55] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. Scaling instruction-finetuned language models. *Journal of Machine Learning Research*, 25(70):1–53, 2024.

[56] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. Stanford alpaca: An instruction-following llama model, 2023.

[57] Keheng Wang, Feiyu Duan, Sirui Wang, Peiguang Li, Yunsen Xian, Chuantao Yin, Wenge Rong, and Zhang Xiong. Knowledge-driven cot: Exploring faithful reasoning in llms for knowledge-intensive question answering. *arXiv preprint arXiv:2308.13259*, 2023.

[58] Haitian Sun, Bhuwan Dhingra, Manzil Zaheer, Kathryn Mazaitis, Ruslan Salakhutdinov, and William Cohen. Open domain question answering using early fusion of knowledge bases and text. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, pages 4231–4242, 2018.

[59] Haitian Sun, Tania Bedrax-Weiss, and William Cohen. Pullnet: Open domain question answering with iterative retrieval on knowledge bases and text. In *Proceedings of the 2019 conference on empirical methods in natural language processing and the 9th international joint conference on natural language processing (EMNLP-IJCNLP)*, pages 2380–2390, 2019.

A. Budgeted Multi-Hop Candidate Subgraph Construction

This section provides a concrete instantiation of a topic-centered rough candidate generator for constructing $\mathcal{G}_q$ when released rough subgraphs are unavailable. In our main experiments, we reuse released rough subgraphs as described in Sec. 4.

Recall the fixed semantic relevance score (Sec. 3):

$$s_{\text{sem}}(\tau) = \cos(z_q,\ z_h + z_r + z_t), \tag{22}$$

where z_q, z_h, z_r, z_t are frozen semantic embeddings produced by a pre-trained text encoder (Sec. 3.2.1). Let $\text{TopK}_K(X;\ s)$ denote the set of K elements in X with the largest scores $s(\tau)$, and let $\mathcal{V}(\cdot)$ denote the incident entity set of a triple set.

Algorithm 2 Budgeted multi-hop expansion for constructing a candidate subgraph $\mathcal{G}_q$

Require: Topic entities $\mathcal{S}_q$; KG triple set $\mathcal{T}$; expansion depth H (e.g., 3); total triple budget M (e.g., 5000); per-hop budgets $\{M^{(l)}\}_{l=1}^{H}$ with $\sum_{l=1}^{H} M^{(l)} = M$ (we use a uniform allocation $M^{(l)} = \lfloor M/H \rfloor$ in practice); fixed score $s_{\text{sem}}(\tau)$.

Ensure: Candidate triple set $\mathcal{G}_q$.

$\mathcal{G}_q^{(0)} \leftarrow \emptyset$, $\mathcal{F}^{(0)} \leftarrow \mathcal{S}_q$
for $l = 1$ to H **do**
  $\mathcal{B}^{(l)} \leftarrow \{\tau = (h, r, t) \in \mathcal{T} : h \in \mathcal{F}^{(l-1)} \vee t \in \mathcal{F}^{(l-1)}\}$
  $\Delta^{(l)} \leftarrow \text{TopK}_{M^{(l)}}(\mathcal{B}^{(l)} \setminus \mathcal{G}_q^{(l-1)};\ s_{\text{sem}})$
  $\mathcal{G}_q^{(l)} \leftarrow \mathcal{G}_q^{(l-1)} \cup \Delta^{(l)}$
  $\mathcal{F}^{(l)} \leftarrow \mathcal{V}(\mathcal{G}_q^{(l)}) \setminus \mathcal{V}(\mathcal{G}_q^{(l-1)})$
return $\mathcal{G}_q \leftarrow \mathcal{G}_q^{(H)}$

B. Candidate-Space Coverage Analysis

This appendix provides diagnostics on candidate-space coverage under the fixed rough subgraphs $\mathcal{G}_q$ and the coarse filtering stage $\mathcal{C}_q$. We report: (i) *Answer-in-$\mathcal{G}_q$ rate*, the fraction of questions whose gold answer set A_q intersects $\mathcal{V}(\mathcal{G}_q)$; (ii) *Path-in-$\mathcal{G}_q$ rate*, the fraction of questions for which at least one topic-to-answer path exists in the undirected view G_q^{und}; (iii) *Retention after coarse retrieval*, measured by the corresponding rates computed on the compact triple subgraph $\tilde{\mathcal{G}}_q = \mathcal{C}_q$ (with incident entity set $\mathcal{V}(\mathcal{C}_q)$); and (iv) size statistics of $|\mathcal{G}_q|$, $|\mathcal{C}_q|$, and $|\mathcal{V}(\mathcal{C}_q)|$.

These diagnostics characterize the upper bound imposed by the candidate protocol and help distinguish retrieval failures from candidate-space limitations.

C. Baseline and Reasoner Settings

We follow the official implementations and released checkpoints for external baselines (RoG, GNN-RAG, DualR, and SubgraphRAG). Since these systems may involve different preprocessing or retrieval paradigms (e.g., path retrieval vs. triple ranking), we report their end-to-end KGQA performance under their official evaluation pipelines. When a baseline supports operating on question-centered rough subgraphs, we provide the same released rough subgraphs used by our retriever; otherwise, we keep the baseline's original KG access protocol.

Unless stated otherwise, we use Llama-3.1-8B-Instruct as the default reasoner with a unified prompt template and temperature-0 decoding. If we report results with API-based reasoners, we specify the exact model IDs, API versioning, and decoding parameters for reproducibility.

D. Full Budget Curves

We provide full curves of retrieval metrics (Answer Entity Recall@K, Shortest-Path Triple Recall@K, GPT-4o Triple Recall@K, and Path Coverage@K) as functions of K, and end-to-end F1/Hit@1 as functions of K under fixed

reasoners. We additionally report sensitivity to the coarse budget K_c.

E. Efficiency Breakdown

We report a breakdown of wall-clock latency across stages (coarse scoring, fine reranking, and total), as well as evidence sizes measured by the number of retrieved triples and the resulting token length after linearization.

F. Additional Ablations and Hyperparameters

We report additional ablations and sensitivity analyses that are omitted from the main text for brevity, including: weak supervision hyperparameters (Δ, P_{near}, B_{type}, α, β), DDE depth L_{DDE}, and SPIMP/ASSF design variants.